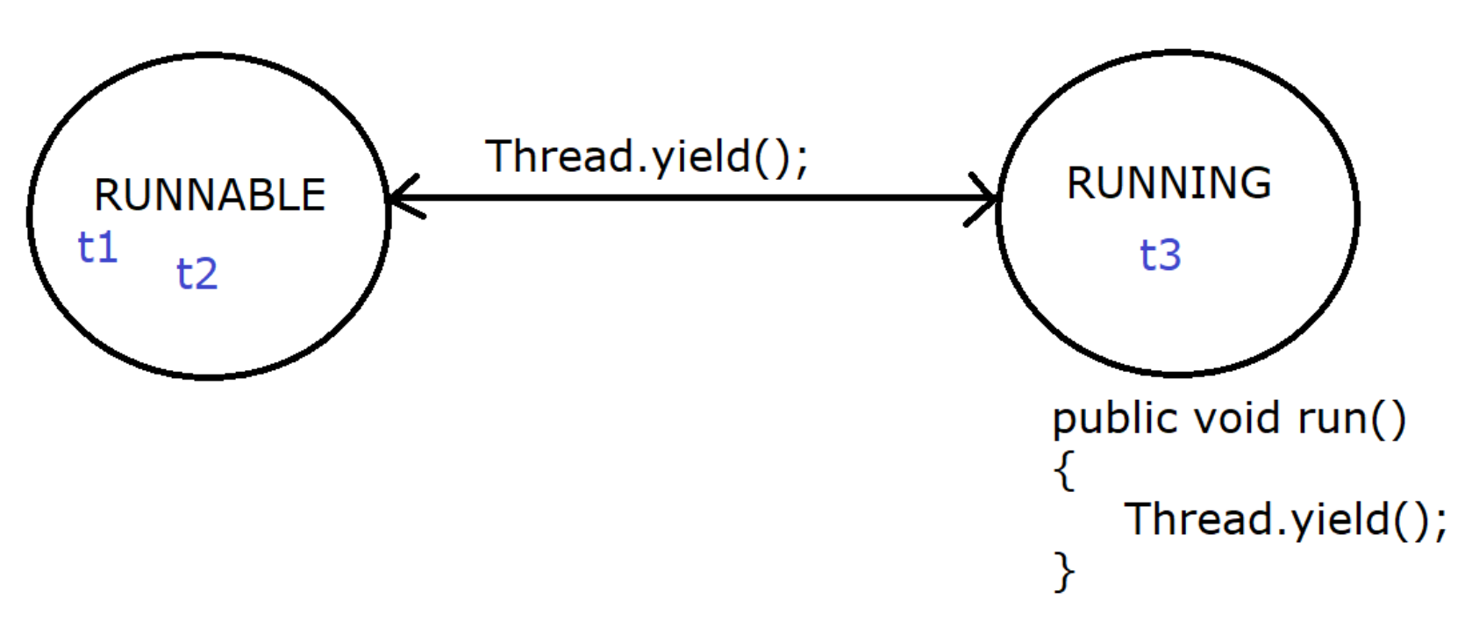
```
package com.ravi.priority;

class Test implements Runnable
{
    @Override
    public void run()
    {
        Thread t = Thread.currentThread();

        int count=0;
        for(int i=1; i<=100000; i++)
        {
            count++;
        }

        IO.println("Completed thread name is :"+t.getName()+" and its priority is
:"+t.getPriority());
    }

}
public class ThreadPriorityDemo3 {

    public static void main(String[] args)
    {
        Test test = new Test();

        Thread t1 = new Thread(test,"First");
        Thread t2 = new Thread(test,"Last");

        t1.setPriority(Thread.MAX_PRIORITY);
        t2.setPriority(Thread.MIN_PRIORITY);

        t1.start();  t2.start();

    }

}
```

Most of time the thread having highest priority will complete its task but we can't say that it will always complete its task first that means Thread schedular dominates Priority of the Thread. Thred schedular is responsible to schedule the thread.

Thread.yield() : [To Prevent a thread from over utilization of CPU]



It is a static method of Thread class.

The currently executing thread will send a notification to thread schedular to stop the currently executing Thread (In Running state) and provide a chance to Threads which are in Runnable state to enter inside the running state having same priority or higher priority than currently executing Thread.

Here The running Thread may directly move from Running state to Runnable state.

The Thread schedular may accept OR ignore this notification message given by currently executing Thread.

Here there is no guarantee that after using yield() method the running Thread will move to Runnable state and the thread which is running thread will move to Runnable state.

If the thread which is in runnable state is having low priority than the current executing thread in Running state,Running thread will continue its execution.

*It is mainly used to avoid the over-utilisation a CPU by the current Thread so all the threads will get equal chance of CPU
for execution as well as In industry we can use this method for testing and debugging purposes.

```
package com.ravi.yield;

class Test implements Runnable
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();

        for(int i=1; i<=10; i++)
        {
            IO.println(i+" by "+name+" thread");

            if(name.equals("Child1"))
            {
                Thread.yield();  //give a chance to Child2
            }
        }
    }
}

public class YieldDemoExample
{
    public static void main(String[] args)
    {
       Test test = new Test();

       Thread t1 = new Thread(test,"Child1");
       Thread t2 = new Thread(test,"Child2");

       t1.start();  t2.start();

    }
}
```

New Thread life Cycle :

A thread is well known for independent execution, During the life cycle of a thread it passes through different states

Java has provided a new Thread life cycle from JDK 1.5V with the support of enum called Thread.State. [Thread $State]

We have a predefined enum called State which is available inside Thread class and it represents life cycle constants which are as follows :

1) NEW (Born State)
2) RUNNABLE (Ready to Run, may be waiting for processor time)
3) BLOCKED (Waiting for Object lock to enter OR reenter in syn area)
4) WAITING (Waiting without any TIMEOUT time , join() wait())
5) TIMED_WAITING (Waiting with TIMEOUT time, sleep(1000), join(1500), wait(1000))
6) TERMINATED (completed the run() method)

NEW :

Whenever we create a thread instance(Thread Object) a thread comes to new state OR born state. New state does not mean that the Thread has started yet only the object or instance of Thread has been created.

RUNNABLE :

Whenever we call start() method on thread object, A thread moves to Runnable state i.e Ready to run state. Here the thread is considered "alive," but it doesn’t immediately start execution unless the CPU scheduler assigns it's time to CPU.

BLOCKED :

If a thread is waiting for object lock OR monitor to enter inside synchronized area OR re-enter inside synchronized area then it is in blocked state.

WAITING :

A thread in the waiting state is waiting for another thread to
perform a particular action but WITHOUT ANY TIMEOUT period.
A thread that has called wait() method on an object is waiting for another thread to call notify() or notifyAll() on the same object OR A thread that has called join() method is waiting for a specified thread to terminate.

TIMED_WAITING :

A thread in the timed_waiting state, if we call any method which put the thread into temporarly timed_waiting state but WITH POSITIVE TIMEOUT period like sleep(lons ms), join(long ms), wait(long ms) then the Thread is considered as Timed_Waiting state.

TERMINATED :

The thread has successuflly completed it's execution in the separate stack memory that means run() method is successfully completed.

**** Inter-Thread Communication :

* It is a communication OR co-ordination **between two synchronized threads** to complete a particular task on the **same object.**

* We can achieve inter-thread communication concept by using **wait(), notify() and notifyAll()** methods of Object class.

Diagram :